BIG DATA ANALYTICS (23AID302)
Final Project Report

ClimaFlu-GNN

Climate-aware influenza forecasting on a Hadoop + Spark pipeline

Group 04
Madhu Hasini — CB.AI.U4AID24060
Saanvika — CB.AI.U4AID24047
Dharshini — CB.AI.U4AID24013

Faculty: SREEJA B P
Amrita Vishwa Vidyapeetham
October 2026

Code and results: https://github.com/vandanapumadhuhasini-ctrl/climate-adaptive-flu-forecasting

Abstract

Influenza-like illness (ILI) rises in each US state at a different time and with a different strength, and public-health agencies need state-level forecasts several weeks ahead. Weather is a known driver of influenza transmission, but the climate record that could inform such forecasts is large and noisy: 26 years of NOAA GHCN-Daily observations amount to 34.6 GB and 968.9 million records. This project builds ClimaFlu-GNN, an end-to-end big data pipeline on a Dockerised Hadoop 3.2.1 cluster with Spark 3.5.1. Raw data is stored in HDFS (three DataNodes), aggregated by two Python MapReduce jobs on YARN, cleaned and joined with CDC ILINet surveillance data by Scala Spark jobs written only with the DataFrame API, and turned into weekly features for 50 states. Forecasts 1–4 weeks ahead are produced by Spark MLlib models (Linear Regression, Random Forest, Gradient-Boosted Trees) and by a graph neural network whose graph links states with similar climate each week. A Structured Streaming job processes a live climate feed, and a Streamlit dashboard presents the results.

On the 2023–2025 test period (7,691 state-weeks), the best models reduce forecast error by 11–17 % relative to persistence. The GNN significantly outperforms a fixed-setting Spark GBT at a 3-week horizon (Holm-adjusted $p = 0.017$), while tuned Random Forest and GBT models perform statistically on par with it. On the big data side, Spark processes the full 34.6 GB 18 times faster than chunked pandas under the same 4 GB memory cap, the stream sustains 38.1 thousand rows per second against 15.3 thousand arriving, and Hadoop MapReduce, Spark and pandas produce byte-identical aggregates.

Contents

1. Introduction 4
1.1 Problem statement 4
1.2 Why big data is needed 4
1.3 Objectives 4
2. Base paper and related work 5
3. Datasets and the 4 V's 6
4. System architecture 7
5. Implementation 8
5.1 HDFS ingestion 8
5.2 MapReduce on YARN (Python, Hadoop Streaming) 8
5.3 Spark ETL in Scala (DataFrame API only) 8
5.4 Spark workflow: DAG, partitioning and caching 9
5.5 Spark MLlib 9
5.6 ClimaFlu-GNN 10
5.7 Structured Streaming 10
5.8 Dashboard 10
6. Evaluation protocol 11
7. Results 12
7.1 Forecast accuracy 12
7.2 Statistical tests 13
7.3 Big data performance 13
8. Challenges and limitations 14
8.1 Challenges 14
8.2 Limitations 14
9. Conclusion and future work 15
References 16
Appendix A. How to run 17
Project structure 17
Appendix B. Cluster evidence 18

1. Introduction

1.1 Problem statement

Seasonal influenza causes a large burden on health systems every winter, but its timing and intensity differ strongly between states. Agencies plan staff, hospital beds and vaccination campaigns with a lead time of weeks, so they need forecasts of weekly ILI activity per state 1–4 weeks ahead. Cold and dry air is associated with higher influenza transmission, which makes daily weather a natural source of predictive information. Using it, however, means processing decades of station-level observations from tens of thousands of weather stations.

1.2 Why big data is needed

- **Volume:** the GHCN-Daily yearly files for 2000–2025 total 34.6 GB and 968.9 million records.
- **Single-machine limits:** an in-memory pandas implementation of the cleaning step ran out of memory on a single year of data under a 4 GB memory cap; a chunked pandas version needed 1,619 s for all years.
- **Distributed tools:** the data must be stored redundantly (HDFS) and processed in parallel (MapReduce, Spark), and new data arrives continuously, which calls for stream processing.

1.3 Objectives

- Store and process at least 25 GB of raw climate data on a Hadoop cluster (HDFS + YARN) running in Docker.
- Clean, aggregate and join climate and influenza data with Scala Spark jobs that use only the DataFrame API.
- Forecast weekly ILI for 50 US states 1–4 weeks ahead and compare Spark MLlib models with a graph neural network under one fair, pre-registered protocol.
- Process a live climate feed with Spark Structured Streaming and present all results on a dashboard.
- Make every number reproducible from result files, with one command that runs every stage.

2. Base paper and related work

The base paper is Y. Zhao *et al.*, "A new two-way ensemble prediction for infectious diseases," IEEE Transactions on Computational Social Systems (2026), which combines several forecasting models for infectious-disease incidence. It does not use climate data and does not address the data-engineering side of forecasting. ClimaFlu-GNN addresses both gaps: it adds 26 years of daily climate observations, processed at scale on Hadoop and Spark, and it tests whether a climate-similarity graph between states improves forecasts.

The link between weather and influenza is well documented; Shaman and Kohn showed that absolute humidity modulates influenza survival and transmission. Graph convolutional networks (Kipf and Welling) provide the building block of the GNN. The processing layer relies on MapReduce (Dean and Ghemawat) and Apache Spark (Zaharia et al.), and forecast comparisons use the Diebold–Mariano test for equal predictive accuracy.

3. Datasets and the 4 V's

Dataset	Source	Size / format	Used for
GHCN-Daily climate	NOAA NCEI	34.6 GB, 26 yearly CSVs (2000–2025), 968.9 M records	daily TMAX, TMIN, PRCP
GHCN stations	NOAA NCEI	fixed-width text; 78,567 US stations kept	station → state
ILINet	CDC FluView	CSV with a title line; 44,808 state-week rows	target: unweighted ILI %
NREVSS (3 tables)	CDC FluView	clinical and public-health laboratory results	laboratory context
Modelling table	own Spark ETL	Parquet; 40,408 state-weeks, 50 states, 2010–2025	every model

Table 1. Datasets used in the project.

- **Volume:** 34.6 GB of raw data in HDFS with replication factor 2 on three DataNodes.
- **Velocity:** a live feed of 60 daily climate files (5,972,783 rows) is processed by Structured Streaming at 38.1 thousand rows per second.
- **Variety:** plain CSV, fixed-width text and CSV files with a free-text title line are converted to Parquet.
- **Veracity:** 1,060,351 quality-flagged observations are removed; the weighted ILI column is 99.85 % missing at state level, so the unweighted ILI rate is used; 827 duplicate New York / New York City state-weeks are merged by summing counts and recomputing the rate.

Key attributes. GHCN: station ID, date, element (TMAX, TMIN, PRCP), value in tenths, quality flag. ILINet: region (state), year, week, ILI total, total patients, unweighted ILI %.

4. System architecture

All components run as Docker containers defined in one docker-compose.yml. The big data layer stores and processes the data; the machine-learning layer trains and evaluates the models on Parquet tables exported from HDFS. Results are written back to HDFS with a SHA-256 manifest.

Container	Role	Web UI
flu-namenode	HDFS NameNode; MapReduce job client	localhost:9870
flu-datanode1/2/3	HDFS DataNodes (replication 2)	localhost:9864–9866
flu-resourcemanager	YARN ResourceManager	localhost:8088
flu-nodemanager	YARN NodeManager with Python 3 for the MapReduce tasks	—
flu-historyserver	YARN application history	localhost:8188
spark	Spark 3.5.1 (Scala jobs, local[*] on HDFS)	localhost:4040 while a job runs
flu-spark-history	Spark History Server: every finished Spark job, stages, DAG, SQL plans	localhost:18080
pytorch	PyTorch Geometric training and evaluation	—
flu-dashboard	Streamlit dashboard (reads result files only)	localhost:8501

Table 2. Docker containers of the cluster.

Data flow. Raw files → HDFS → (a) MapReduce on YARN for raw-data aggregates and (b) Spark ETL → modelling table and climate graph → Spark MLlib and the GNN → evaluation → results, dashboard and HDFS. In parallel, Structured Streaming reads daily files dripped into an HDFS folder.

One entry point. run_pipeline.ps1 runs every stage in order (up, ingest, etl, mapreduce, gnn, v3, mllib, streaming, performance, checks, report, publish) or a single Spark job by name, e.g. .\run_pipeline.ps1 job Preprocess.

5. Implementation

5.1 HDFS ingestion

The raw files are bind-mounted read-only into the NameNode container and copied with hdfs dfs -put into /flu_project/raw. HDFS stores 34,586,251,521 bytes of raw data; fsck reports the file system as HEALTHY with no under-replicated blocks.

5.2 MapReduce on YARN (Python, Hadoop Streaming)

Both MapReduce jobs are written in Python (climaflu_mapreduce.py) and submitted to YARN with the Hadoop Streaming jar; the NodeManager image adds Python 3. Each input line passes the same filter used by the Spark and pandas jobs: US station, element TMAX/TMIN/PRCP, empty quality flag, integer value.

- **MR1 (element × year):** the mapper emits (element|year, 1, value); the reducer adds counts and sums and is also used as the combiner. 268 map tasks read all 968,905,773 records in 1,290 s and produce 78 groups.
- **MR2 (state × year × element):** a map-side join: the stations file is shipped to every mapper through the distributed cache, and observations are joined to their state in the mapper. It produces 3,920 groups in 1,096 s.
- **Record accounting:** Hadoop counters show KEPT 303,631,696 + NOT_US 305,732,176 + OTHER_ELEMENT 358,481,524 + QC_FLAGGED 1,060,351 + HEADER 26 = 968,905,773 input records exactly.

5.3 Spark ETL in Scala (DataFrame API only)

Every Spark job uses the DataFrame API; there is no RDD or typed Dataset code, and an automated scan of the code base confirms it. The ETL jobs are:

- **Preprocess:** parses the fixed-width station file with substring and trim, and cleans the four CDC FluView CSVs (title line removed, types cast, missing markers set to null).
- **WeatherPreprocess:** reads all 26 yearly CSVs with a fixed schema, keeps US TMAX/TMIN/PRCP without quality flag, scales values and writes 303.6 M rows as Parquet partitioned by element.
- **BuildModelingTable:** aggregates daily climate to MMWR state-weeks (pivot of TMAX, TMIN, PRCP plus diurnal range) and joins it with ILINet, merging New York and New York City.
- **GraphConstruction:** builds the weekly climate-similarity KNN graph between states.
- **LagAwareFeaturesV3:** builds lagged, lead-aligned climate features and targets for 1 to 4 weeks ahead.

```
val cleaned = spark.read.schema(weatherSchema).csv(s"$rawDir/*.csv")
  .filter(c("id").startsWith("US"))
  .filter(c("element").isin("TMAX", "TMIN", "PRCP"))
  .filter(c("qflag").isNull || c("qflag") === "")
  .withColumn("obs_date", to_date(c("date"), "yyyyMMdd"))
  .withColumn("value_scaled", c("value") / 10.0)
cleaned.write.mode("overwrite").partitionBy("element")
  .parquet(s"$processedDir/weather_cleaned")
```

Listing 1. Cleaning step of WeatherPreprocess.scala (c = col).

5.4 Spark workflow: DAG, partitioning and caching

Spark builds a DAG of stages from these transformations; a new stage starts at every shuffle (groupBy, join). The 78,567-row station table is broadcast, so the 303.6 M weather rows are joined without a shuffle. A controlled experiment on the project's own feature step compared partition counts (8, 24, 96, 384) with and without persistence, three repeats each.

Partitions	No persistence (s)	persist(MEMORY_AND_DISK) (s)
8	510	1,397
24	363	out of memory
96	346	724
384	326	702
Spark defaults with AQE (200)	285	—

Table 3. Total time of three queries on 303.6 M rows (median of three runs).

Adaptive Query Execution with default settings was fastest. Caching did not help: with a 3 GB driver heap the cached data spilled to disk, and at 24 partitions it ran out of memory. The pipeline therefore recomputes instead of caching large frames, and caches only small results (for example the 68,833-row state-week table).

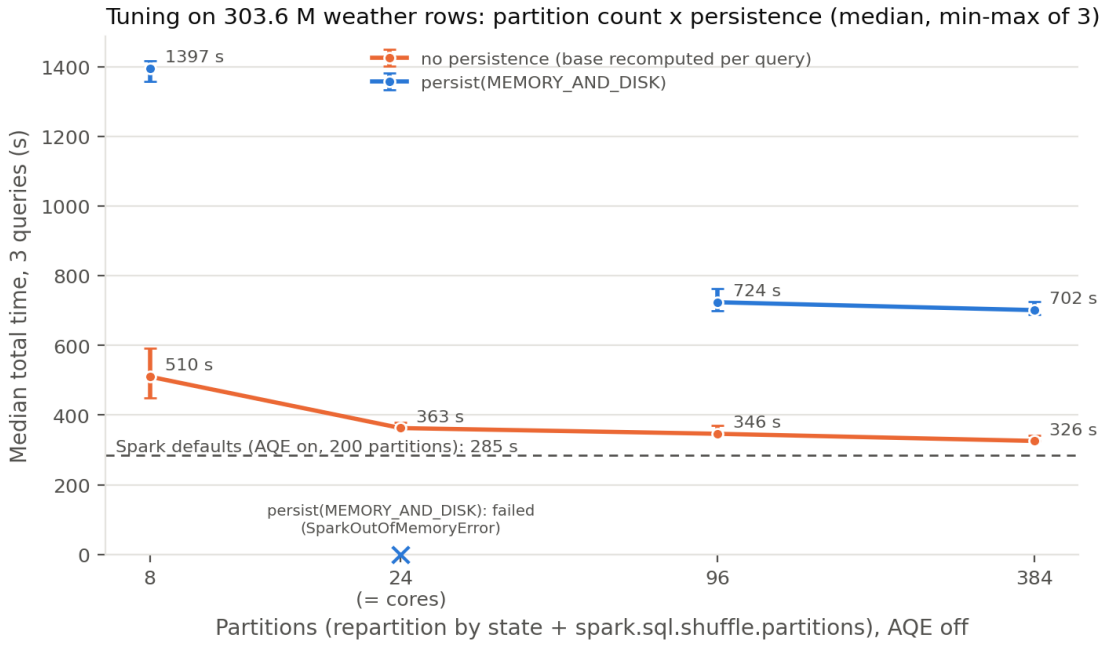


Figure 1. Partitioning and persistence experiment.

5.5 Spark MLlib

MLlib Pipelines combine a VectorAssembler with a regressor. Linear Regression, Random Forest and GBT are trained on two feature sets: ILI lags only (last four weeks plus trend) and lags plus climate (weekly TMAX, TMIN, PRCP and diurnal range). Hyperparameters are tuned with ParamGridBuilder grids (for example Random Forest trees {50, 100, 200} × depth {5, 10}); the best setting is chosen on the 2022 validation year with a RegressionEvaluator (RMSE), never on the test years. Fixed-setting GBT baselines (50 iterations, depth 5) are also trained.

5.6 ClimaFlu-GNN

Each week a graph is built that links every state to its eight most climate-similar states (K = 8), using statistics from the training period only. The model has two branches: a two-layer graph convolution over the weekly climate features and an MLP over the ILI history; their outputs are fused by a small MLP. Hidden size 16, Adam (learning rate 0.001, weight decay 0.00001), gradient clipping 1.0, batch size 8, early stopping on 2022 (at most 200 epochs, patience 30), then a refit on all data up to 2022. Each model is trained with 10 seeds. Ablations replace the graph with no graph, a random graph, a static graph and a geographic graph.

5.7 Structured Streaming

For velocity, the 2025 climate data is split into 60 daily files. A drip script moves one file every 3 s into a watched HDFS folder (atomic rename). The streaming query uses a 5 s trigger, filters and joins each micro-batch to the broadcast station table, and appends state × day × element aggregates to HDFS. A listener logs every micro-batch to measure throughput and latency.

5.8 Dashboard

A Streamlit dashboard (localhost:8501) shows the 4 V's, cluster health, MapReduce and Spark results, the benchmark and tuning experiments, streaming metrics, forecasts and the climate graph. It reads result files only, through read-only mounts.

6. Evaluation protocol

- **Temporal split:** training ≤ 2022 (31,674 rows), with 2022 (2,594 rows) used for model selection; test 2023–2025 (7,691 state-weeks, 156 weeks). Every model is evaluated on the identical test rows.
- **Metric:** root mean squared error (RMSE) in ILI percentage points.
- **Significance:** Diebold–Mariano test with Newey–West variance and the Harvey–Leybourne–Newbold correction, Holm correction across horizons, and a block bootstrap confidence interval.
- **Pre-registration:** splits, metrics, seeds and decision rules were fixed in docs/RESEARCH_PROTOCOL.md before the runs; later additions are recorded as amendments.

7. Results

7.1 Forecast accuracy

Test RMSE (ILI %)	1 week	2 weeks	3 weeks	4 weeks
Persistence (last week)	1.038	1.303	1.479	1.632
Spark Linear Regression (tuned)	0.992	1.235	1.378	1.530
Spark Random Forest (tuned)	0.924	1.131	1.267	1.366
Spark GBT (tuned)	0.929	1.130	1.267	1.382
Spark GBT (fixed setting)	0.938	1.154	1.289	1.382
ClimaFlu-GNN (mean of 10 seeds)	**0.924**	**1.145**	**1.251**	**1.354**

Table 4. Test RMSE by forecast horizon (MLlib models with climate features).

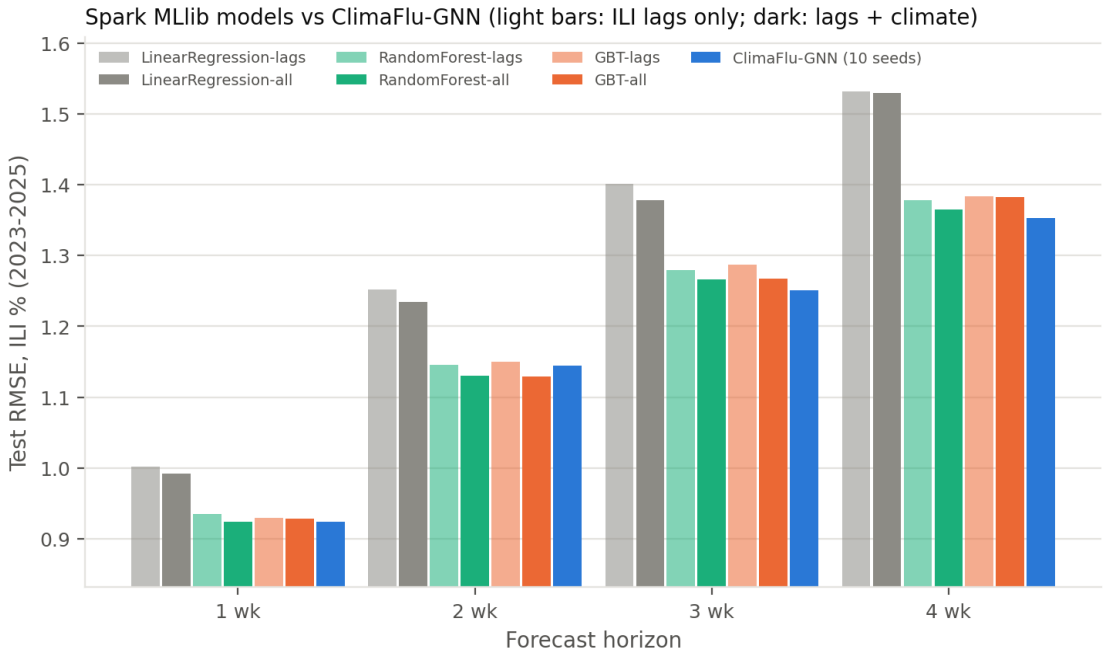


Figure 2. Spark MLlib models and ClimaFlu-GNN by horizon.

- The GNN and the tree models reduce error relative to persistence by 11 % at one week and 17 % at four weeks.
- The GNN has the lowest error at 3 and 4 weeks and ties the tuned Random Forest at one week.
- Adding climate features lowers Random Forest error at every horizon (for example 1.280 → 1.267 at three weeks).
- Linear Regression is clearly weaker; its lag coefficients are unstable because the trend feature is a linear combination of the first two lags.

7.2 Statistical tests

Claim (Holm-adjusted)	Result
GNN better than the fixed-setting Spark GBT	supported at 3 weeks (p = 0.017); not significant at 1, 2 and 4 weeks
GNN better than the best tuned MLlib model	not supported at any horizon (p ≥ 0.18)
Climate improves the neural model at longer horizons	not significant (p = 0.079 at 3 and 4 weeks)
Climate graph better than no graph / random graph	not supported

Table 5. Pre-registered claims and outcomes.

Reporting the unsupported claims is part of the protocol: the evidence shows that a well-tuned tree model with climate features is as accurate as the GNN, and that the benefit of the climate graph itself is not statistically significant.

7.3 Big data performance

Years of data	1	3	5	10	26
Spark, HDFS, 24 cores (s)	7.6	16.5	20.9	38.4	90.1
pandas, chunked (s)	55.2	163.9	279.8	590.5	1,619.0

Table 6. Same workload and result checksum, each run in a fresh container with a 4 GB memory cap.

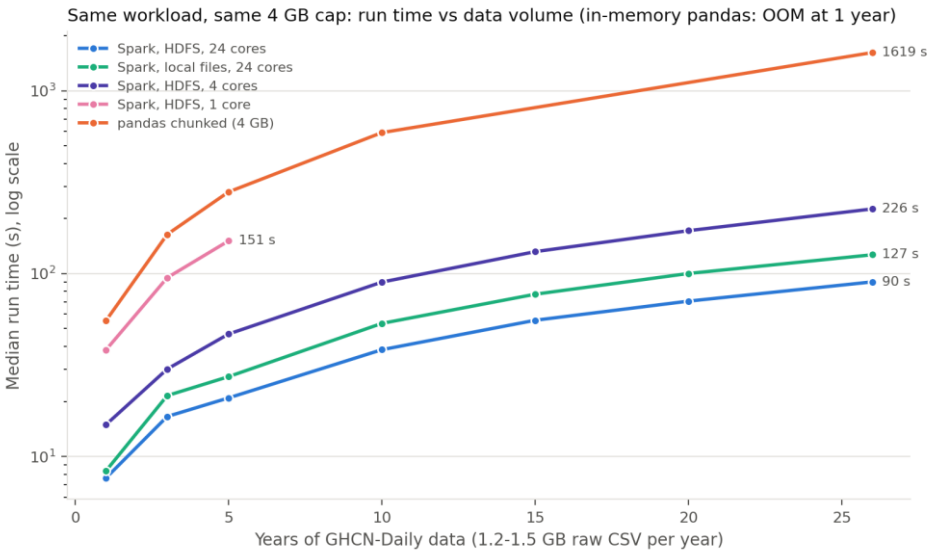


Figure 3. Scalability benchmark (log scale).

- On all 26 years Spark is 18 times faster than chunked pandas; in-memory pandas fails already at one year.
- Streaming: 5,972,783 rows in 59 micro-batches; 38.1 thousand rows/s processed against 15.3 thousand rows/s arriving; latency from file arrival to committed output median 4.7 s (95th percentile 7.6 s); every file processed exactly once.
- Consistency: MR1 and the Spark DataFrame job give the same SHA-256 (a5c81104…) as the pandas benchmark, and MR2 matches the Spark broadcast join (eb02de06…).

8. Challenges and limitations

8.1 Challenges

- **Memory:** Hadoop and Spark share one 8 GB Docker VM; drivers above 3 GB (2 GB for the largest aggregation) were killed by the Linux out-of-memory killer, so memory settings were tuned per job.
- **Disk:** Spark shuffle files inside the container permanently grew the Docker disk image; shuffle storage was moved to a host folder and long jobs run under a disk watchdog.
- **Messy public-health data:** CSV title lines, duplicate New York / New York City rows and an almost empty weighted ILI column required explicit cleaning rules.
- **Reproducibility:** every stage writes a JSON result file, and every document is generated from those files; MapReduce, Spark and pandas outputs are compared by checksum.

8.2 Limitations

- All containers run on a single machine, so HDFS replication and YARN scheduling do not reflect a real multi-node cluster (all map tasks are rack-local).
- Spark runs in local mode on HDFS rather than on YARN.
- The climate-similarity graph did not give a statistically significant improvement over a model without a graph.

9. Conclusion and future work

ClimaFlu-GNN delivers a complete and reproducible big data pipeline: 34.6 GB of climate data in HDFS, MapReduce jobs on YARN, Scala Spark ETL with the DataFrame API, Spark MLlib, a graph neural network, Structured Streaming and a dashboard, all started by one command. Forecasts are 11–17 % more accurate than persistence 1–4 weeks ahead, and the GNN beats the fixed-setting Spark GBT at three weeks, while tuned tree models perform on par with it. Spark processes the full data 18 times faster than pandas, and all engines produce identical aggregates.

- Run the cluster on several machines and submit Spark jobs to YARN.
- Add humidity, vaccination coverage and mobility data, which are known drivers of influenza.
- Ingest the weekly CDC release as a live feed and add forecast intervals.

References

[1] Y. Zhao et al., "A new two-way ensemble prediction for infectious diseases," IEEE Transactions on Computational Social Systems, vol. 13, no. 5, pp. 6359–6370, 2026, doi:10.1109/TCSS.2025.3640950.

[2] M. J. Menne, I. Durre, R. S. Vose, B. E. Gleason and T. G. Houston, "An overview of the Global Historical Climatology Network-Daily database," Journal of Atmospheric and Oceanic Technology, vol. 29, pp. 897–910, 2012.

[3] Centers for Disease Control and Prevention, FluView Interactive: ILINet and NREVSS surveillance data.

[4] J. Shaman and M. Kohn, "Absolute humidity modulates influenza survival, transmission, and seasonality," Proceedings of the National Academy of Sciences, vol. 106, no. 9, pp. 3243–3248, 2009.

[5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," International Conference on Learning Representations (ICLR), 2017.

[6] M. Zaharia et al., "Apache Spark: a unified engine for big data processing," Communications of the ACM, vol. 59, no. 11, pp. 56–65, 2016.

[7] J. Dean and S. Ghemawat, "MapReduce: simplified data processing on large clusters," USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2004.

[8] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," Journal of Business & Economic Statistics, vol. 13, no. 3, pp. 253–263, 1995.

Appendix A. How to run

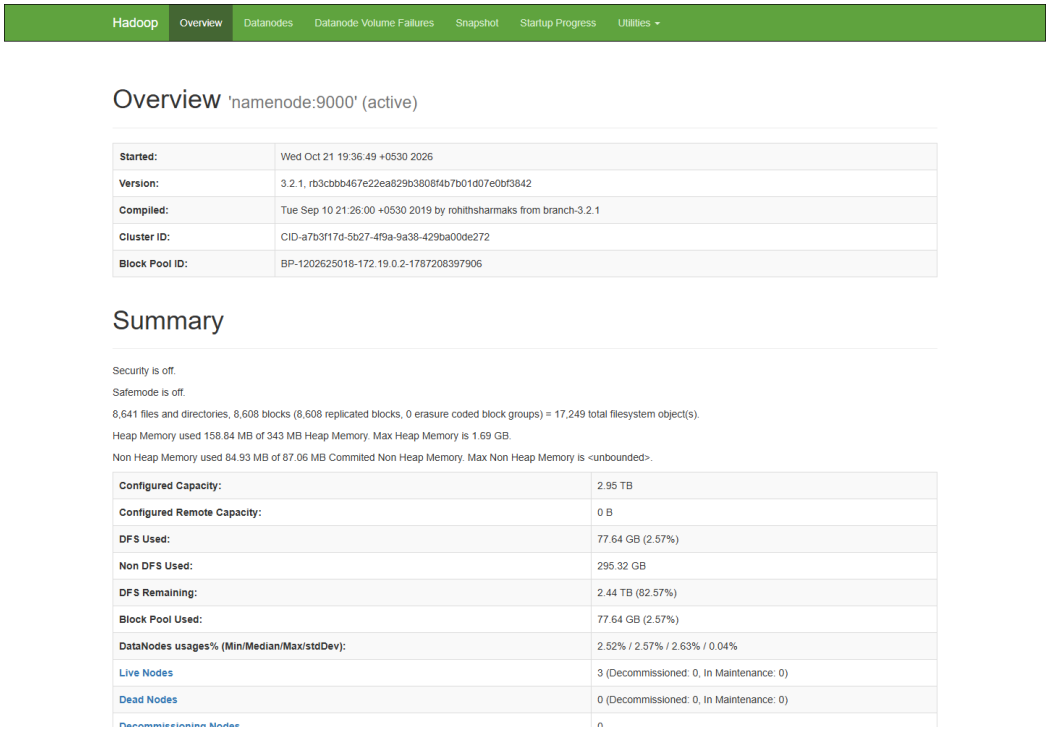
```
docker compose up -d                        # or: .\run_pipeline.ps1 up
.\run_pipeline.ps1                           # list the stages
.\run_pipeline.ps1 etl                       # one stage
.\run_pipeline.ps1 all                       # every stage in order
.\run_pipeline.ps1 job Preprocess            # one Spark job by name
```

Project structure

Folder / file	Contents
run_pipeline.ps1	the one entry point for every stage
docker-compose.yml, hadoop.env, docker/	cluster definition and images
bigdata_pipeline/hadoop_mapreduce/	Python MapReduce jobs and their run script
bigdata_pipeline/spark_etl/	Preprocess, WeatherPreprocess, BuildModelingTable, GraphConstruction, LagAwareFeaturesV3
bigdata_pipeline/spark_mllib/	BaselineGBT, MLlibComparison
bigdata_pipeline/spark_streaming/	StreamingV2 and the drip-feed script
bigdata_pipeline/spark_tuning/	benchmark workload (Spark and pandas), tuning experiment
bigdata_pipeline/spark_checks/	DataChecks, DataFrameConsistency, PublishResultsToHdfs
bigdata_pipeline/dashboard/	Streamlit dashboard
src/gnn/, src/reporting/	GNN training and evaluation; analysis and evidence documents
docs/, results/	protocol, results, figures, screenshots, this report and the presentation

Table 7. Repository structure.

Appendix B. Cluster evidence



Hadoop | Overview | Datanodes | Datanode Volume Failures | Snapshot | Startup Progress | Utilities

Overview 'namenode:9000' (active)

Started:	Wed Oct 21 19:36:49 +0530 2026
Version:	3.2.1, rb3cbbb467e22ea829b3808f4b7b01d07e0bf3842
Compiled:	Tue Sep 10 21:26:00 +0530 2019 by rohithsharmaks from branch-3.2.1
Cluster ID:	CID-a7b3f17d-5b27-4f9a-9a38-429ba00de272
Block Pool ID:	BP-1202625018-172.19.0.2-1787208397906

Summary

Security is off.

Safemode is off.

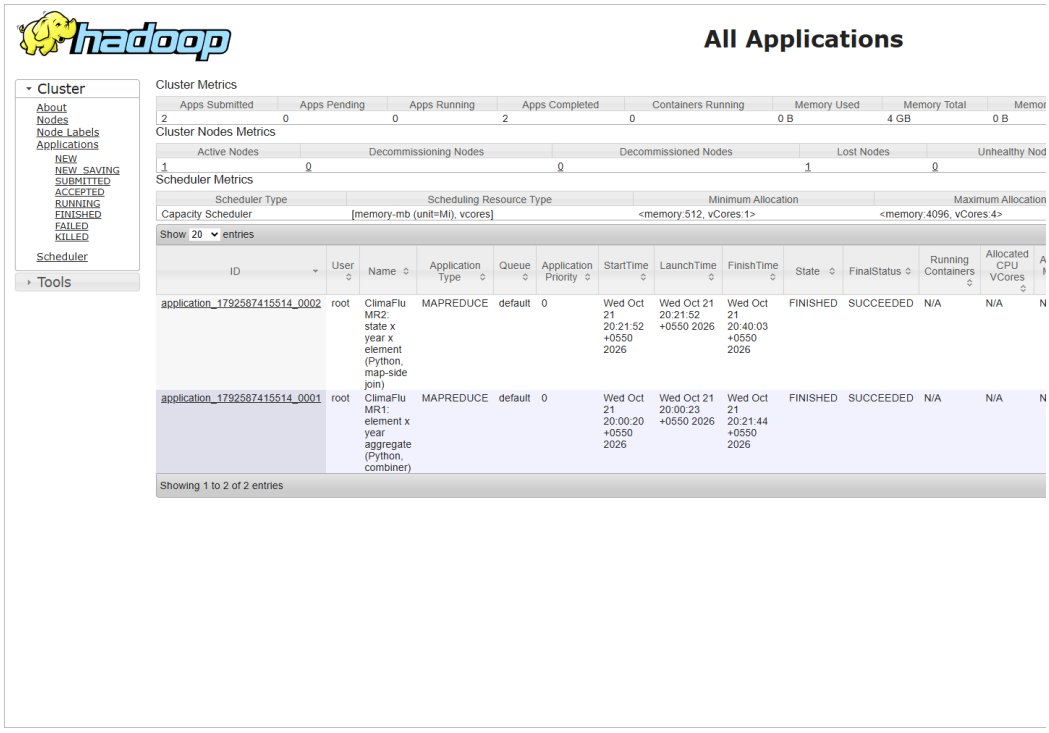
8,641 files and directories, 8,608 blocks (8,608 replicated blocks, 0 erasure coded block groups) = 17,249 total filesystem object(s).

Heap Memory used 158.84 MB of 343 MB Heap Memory. Max Heap Memory is 1.69 GB.

Non Heap Memory used 84.93 MB of 87.06 MB Commited Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	2.95 TB
Configured Remote Capacity:	0 B
DFS Used:	77.64 GB (2.57%)
Non DFS Used:	295.32 GB
DFS Remaining:	2.44 TB (82.57%)
Block Pool Used:	77.64 GB (2.57%)
DataNodes usages% (Min/Median/Max/stdDev):	2.52% / 2.57% / 2.63% / 0.04%
Live Nodes	3 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0

Figure 4. HDFS NameNode (localhost:9870): three live DataNodes, safe mode off.



hadoop

All Applications

- Cluster
  - About
  - Nodes
  - Node Labels
  - Applications
    - NEW
    - NEW_SAVING
    - SUBMITTED
    - ACCEPTED
    - RUNNING
    - FINISHED
    - FAILED
    - KILLED
  - Scheduler
- Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Memory Used	Memory Total	Memor
2	0	0	2	0	0 B	4 GB	0 B

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nod
1	0	0	1	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:512, vCores:1>	<memory:4096, vCores:4>

Show 20 entries

ID	User	Name	Application Type	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU VCores
application_1792587415514_0002	root	ClimaFlu MR2: state x year x element (Python, map-side join)	MAPREDUCE	default	0	Wed Oct 21 20:21:52 +0550 2026	Wed Oct 21 20:21:52 +0550 2026	Wed Oct 21 20:40:03 +0550 2026	FINISHED	SUCCEEDED	N/A	N/A
application_1792587415514_0001	root	ClimaFlu MR1: element x year aggregate (Python, combiner)	MAPREDUCE	default	0	Wed Oct 21 20:00:20 +0550 2026	Wed Oct 21 20:00:23 +0550 2026	Wed Oct 21 20:21:44 +0550 2026	FINISHED	SUCCEEDED	N/A	N/A

Showing 1 to 2 of 2 entries

Figure 5. YARN (localhost:8088): both Python MapReduce jobs finished successfully.